

Application Seedwork 復習ガイド

DDDで「アプリケーション層って何をする場所？」を、できるだけ噛み砕いて理解するための復習ガイドです。

ひとことと言うと

Application層は、「業務をどういう順番で動かすか」を指揮する場所です。

Domainが「何が正しいか」を決めるのに対して、Applicationは「誰が、何を、どの順番で呼ぶか」を決めます。

1. まず3つの層を覚えよう

DDDで迷ったら、まず次の3つに分けて考えます。

Domain	Application	Infrastructure
「何が正しい？」	「どう動かす？」	「どうやって接続する？」
業務ルール・Entity・Value Object・Aggregate	Use Case・Command・Query・DTO・UoW	DB・SQLAlchemy・外部API・具体実装

覚え方：Domain＝ルール、Application＝手順、Infrastructure＝技術

2. Application層は「司令塔」

たとえば「会員を登録する」という仕事を考えます。

Application層自身が「名前は50文字以内」などのルールを判断するではありません。Domainに「会員を作ってください」とお願いし、その結果を保存する、という順番を管理します。

イメージすると、Application層は現場監督です。自分で作業するのではなく、適切な担当者へ仕事を振ります。

3. Use Case（ユースケース）

Use Caseとは、「利用者がやりたい1つの仕事」です。

- ・会員を登録する
- ・注文を確定する
- ・商品を変更する
- ・会員情報を取得する

基本的には、1つのユーザー操作・業務操作を1つのUse Caseとして考えると分かりやすくなります。

4. CommandとQuery

Commandは「状態を変えてください」というお願いです。

例：会員登録、注文確定、商品変更。

Queryは「情報を見せてください」というお願いです。

例：会員検索、注文詳細取得、商品一覧取得。

Command	Query
状態を変更する	基本的に状態を変更しない
登録・更新・削除	検索・取得・一覧
execute()	handle()

5. DTO（データを運ぶ箱）

DTOは、データを受け渡すための箱です。

「会員登録してください」という情報をCommandとして運んだり、「登録された会員」をResponseとして返したりするときに使います。

DTOに業務ルールを詰め込みすぎないことが大切です。

6. Result（結果を入れる箱）

Use Caseの結果を「成功したのか」「失敗したのか」と一緒に返すための仕組みです。

成功なら `Result.ok(...)`、失敗なら `Result.fail(...)` と考えると簡単です。

7. Repositoryは「保存係」、Applicationは「お願いする係」

Application層は「SQLをどう書くか」を知りません。

Applicationは「この会員を保存してください」とRepositoryの抽象へお願いするだけです。実際にSQLAlchemyで保存する方法はInfrastructure層が担当します。

Application	Infrastructure
「会員を保存して」	「SQLAlchemyで保存します」
Repositoryを利用	SQLAlchemyRepositoryを実装

8. Unit of Work（まとめて成功・まとめて失敗）

複数の保存処理をするとき、「途中まで保存されてしまう」のを防ぐための仕組みです。

たとえば「注文を確定して、在庫を更新する」という処理なら、両方成功したときだけ確定し、途中で失敗したら元に戻したいことがあります。

Applicationは `IUnitOfWork` という抽象を使い、実際の `commit / rollback`はInfrastructureに任せます。

覚え方：「全部成功したらOK。途中で失敗したらなかったことにする」

9. Identity（「誰が操作している？」）

Identityは、現在の利用者や実行主体を表します。

Applicationは「HTTPのヘッダーからユーザーを取得する」といった具体的な方法を知りません。`IdentityContext`を通して「今の利用者は誰？」と問い合わせます。

10. Domain Eventとの関係

Domainが「注文が確定した」という重要な出来事をDomain Eventとして発生させることがあります。

Applicationは、そのイベントをAggregateから回収してPublisherへ渡すなど、イベントを適切なタイミングで配送する調整役になります。

イベントの「業務上の意味」はDomainに置き、Applicationが新しい業務ルールを作らないようにします。

11. Application Service

Application Serviceは、複数の処理をまとめてユースケースを動かすためのクラスです。

イメージは次の通りです。

Command → Application Service → Repository → Domain → Repository → Unit of Work → Result

12. 具体例：「注文を確定する」

利用者が「注文確定」ボタンを押したとします。

- ① ApplicationがCommandを受け取る。
- ② Repositoryに「この注文をください」とお願いする。
- ③ DomainのOrderに「確定してください」とお願いする。
- ④ Order自身が業務ルールを確認する。
- ⑤ 問題なければRepositoryへ保存をお願いする。
- ⑥ Unit of Workでcommitする。
- ⑦ Domain EventがあればPublishする。
- ⑧ Resultを返す。

ここで重要なのは、Applicationが「注文確定のルール」を判断していないことです。Applicationは順番を管理しているだけです。

13. やってはいけないこと

- Application Serviceに「金額が0円未満ならエラー」のような業務ルールを書く。
- ApplicationからSQLAlchemy Sessionを直接操作する。
- ApplicationからSQLを直接実行する。
- DB ModelをそのままDomainへ渡す。
- Entityの属性を外から直接書き換える。
- HTTP RequestやFastAPIなどの具体技術にUse Caseを強く依存させる。

迷ったら「これは業務ルール？ 手順？ 技術？」と考えます。

業務ルール→Domain、手順→Application、技術→Infrastructureです。

14. このSeedworkで覚えておきたい部品

部品	ひとことで
Command	「状態を変えてください」
Query	「情報をください」
IUseCase	「この仕事を実行します」
IQueryHandler	「この検索を処理します」
DTO	「データを運ぶ箱」
Result	「成功・失敗の結果」
IUnitOfWork	「まとめて成功・まとめて失敗」
IIidentityContext	「今、誰が操作している？」
ApplicationService	「処理の順番を指揮する」
Bus	「Command / Queryを担当者へ届ける」

15. AIと一緒に実装するときの考え方

AIに「会員登録機能を作って」と頼む前に、次の順番で考えるとDDDの境界を守りやすくなります。

- ・① これは状態変更？ → Command
- ・② これは読み取り？ → Query
- ・③ どのDomainモデルが仕事を担当する？
- ・④ Repositoryは何か必要？
- ・⑤ Unit of Workは必要？
- ・⑥ 入力・出力はどんなDTO？
- ・⑦ エラーはDomain / Application / Infrastructureのどこ？
- ・⑧ 具体的なDBや外部APIはInfrastructureへ隠せている？

AIに相談するときは、「まず設計を提案して、その後に実装する」という2段階にすると、責務の混在を防ぎやすくなります。

16. 最後に覚える3つ

- ① Domainはルール。
「何が正しいか」を決める。
- ② Applicationは手順。
「誰に何を頼んで、どの順番で仕事を進めるか」を決める。
- ③ Infrastructureは技術。
「DBや外部APIなどを、実際にどう接続するか」を担当する。

この3つを守れば、Application層は「何でも入れる便利クラス」ではなく、DomainとInfrastructureをつなぐ明確なユースケース層として設計できます。